Back-end

Documentação técnica - Prumo

API HTTP em Node.js + Express (módulos ES). Código em `backend/`. Referência dos endpoints: API/README.md.

Tecnologias

Pacote	Uso
express 4	Servidor HTTP e rotas
pg 8	Conexão PostgreSQL (consultas parametrizadas)
jsonwebtoken 9	Emissão e validação do JWT (8 horas)
bcryptjs 2	Hash de senhas (custo 10)
multer 2	Recebimento de PDF e vídeo (em arquivo temporário, apagado ao fim da requisição)
@aws-sdk/client-s3, @aws-sdk/s3-request-presigner 3	Bucket compatível com S3 (Cloudflare R2): upload direto e URLs assinadas
@supabase/supabase-js 2	Armazenamento de arquivos no Supabase (legado)
compression	Compactação gzip das respostas JSON
cors, dotenv, uuid	CORS, variáveis de ambiente, nomes de arquivo
nodemon (dev)	Recarga automática em `npm run dev`

Estrutura

```
backend/
+-- sql/schema.sql          estrutura do banco (ver ../Banco de Dados/README.md)
+-- src/
|   +-- server.js           carrega .env, abre a porta e desliga sem cortar requisições (SIGTERM)
|   +-- app.js              monta Express: proxy, gzip, CORS, JSON (1 MB), /uploads com cache, /health, /
health/ready, rotas, 404 e erros
|   +-- routes/             define URL + middlewares de cada endpoint
|   +-- controllers/        valida a requisição e coordena a operação
|   +-- models/             SQL de cada entidade
|   +-- middleware/         auth.js, upload.js, asyncHandler.js
|   +-- config/             db.js, storage.js, objectStorage.js (S3/R2), demoStore.js
|   +-- utils/security.js   política de senha e cálculo da nota
+-- test/security.test.js   testes (node --test)
+-- uploads/                arquivos locais (ignorado pelo Git)
```

Fluxo de uma requisição: `routes` -> `authenticate` -> verificação de perfil -> (`upload`) -> `controller` -> `model` -> PostgreSQL.

Rotas montadas

Prefixo	Arquivo de rotas	Documentação
`/health`	`app.js`	API/HEALTH.md
`/api/auth`	`routes/auth.js`	API/AUTH.md
`/api/users`	`routes/users.js`	API/USERS.md
`/api/sectors`	`routes/sectors.js`	API/SECTORS.md
`/api/systems`	`routes/systems.js`	API/SYSTEMS.md
`/api/manuals`	`routes/manuals.js`	API/MANUALS.md
`/api/progress`	`routes/progress.js`	API/PROGRESS.md
`/api/exams`	`routes/exams.js`	API/EXAMS.md
`/api/uploads`	`routes/uploads.js`	API/UPLOADS.md
`/api/branding`	`routes/branding.js` (corpo até 2 MB)	API/BRANDING.md
`/uploads`	`app.js` (arquivos estáticos)	API/MANUALS.md

Middlewares

Middleware	Regra
`authenticate`	Exige `Authorization: Bearer <token>` válido; grava o usuário do token em `req.user` (`id`, `nome`, `role`) e acrescenta `sector_ids`, lido de `user_sectors` a cada requisição
`requireSupport`	Perfis `suporte` (ou legado `gerente`)
`requireManualManager`	Perfis `suporte`, `editor_conteudo`, `gerente_setor` (ou legado `gerente`)
`uploadManualFiles`	Campos `pdf` (até 20 MB) e `video` (até 200 MB), um de cada; aceita PDF, MP4, WebM e QuickTime. Grava em arquivo temporário e apaga ao fim da requisição; acima do limite responde `413`
`asyncHandler`	Encaminha erros assíncronos ao tratador final

O perfil fica no token e vale até ele expirar; mudar o perfil de um usuário só tem efeito no próximo login. Os setores são relidos a cada requisição e valem na hora. As regras de setor compartilhadas ficam em `utils/access.js` (`seesAllSectors`, `hasSector`, `parseSectorIds`).

Armazenamento de arquivos (`config/storage.js`)

O destino é escolhido por `STORAGE_DRIVER` ou, sem ele, detectado pelo `.env` (nesta ordem):

- `s3` (com `S3_BUCKET`): bucket privado compatível com S3, usado no piloto com Cloudflare R2. O navegador envia direto ao bucket por URL assinada (API/UPLOADS.md); o banco guarda só a chave do objeto (`pdfs/<uuid>.pdf`, `videos/<uuid>.mp4`) e a leitura usa URL assinada temporária, gerada depois de conferir o acesso. Código em `config/objectStorage.js`.
- `supabase` (com `SUPABASE_URL` e `SUPABASE_SERVICE_ROLE_KEY`): envia ao bucket e grava a **URL pública** do arquivo.
- `local`: grava em `backend/uploads/pdfs` ou `backend/uploads/videos` e grava o caminho `/uploads/...`. Só para desenvolvimento.

PDFs passam por uma limpeza que neutraliza comandos de impressão automática embutidos. Nos modos `supabase` e `local`, o arquivo chega em disco temporário (não na memória) e é apagado ao fim da requisição.

O plano para Google Cloud (bucket privado e URLs assinadas) está em Geral/IMPLANTACAO-GOOGLE-CLOUD.md.

Tratamento de erros

- Rota inexistente: `404 { "message": "Rota não encontrada" }`.
- Erros de conexão com o banco (senha errada, conexão recusada, banco inexistente): `500` com "Banco de dados indisponível ou credenciais incorretas".
- Demais erros: status do erro (ou `500`) com a mensagem da exceção.

Modo demonstração (`config/demoStore.js`)

Quando `DEV_DEMO_MODE=true` e o PostgreSQL está fora do ar, login, usuários, setores, sistemas, manuais, progresso e provas usam dados em memória (usuário `admin`/`admin123`). A auditoria não tem fallback. Uso exclusivo em desenvolvimento.

Testes

`npm test` executa `test/security.test.js` (política de senha) e `test/exam.test.js` (validação do cadastro de prova, período de realização e correção das respostas, em `utils/exam.js`). Não há testes de rotas nem de banco.

Limitações conhecidas

- A API aceita `pdfCompleted` e `videoProgress` enviados pelo navegador sem conferir (ver REGRAS-DE-NEGOCIO.md).
- Nos modos `local` e `supabase`, arquivos abrem sem login para quem tiver o link. No modo `s3`, a URL assinada também abre sem login enquanto não expira (padrão 4 horas).
- Sem `JWT_SECRET` o servidor inicia, mas todo login falha.
- No Cloud Run, requisições acima de 32 MB são recusadas antes de chegar ao backend; vídeos maiores dependem do upload direto ao Cloud Storage (ver IMPLANTACAO-GOOGLE-CLOUD.md).
- Excluir manual não apaga os arquivos do armazenamento (o banco perde a referência a eles).
- O bloqueio de login fica em memória e não é compartilhado entre instâncias.
- Mensagens de erro internas são devolvidas ao cliente.